

AGENTE DE ANÁLISE EXPLORATÓRIA DE DADOS

Aplicação sobre o Dataset F1 Race Results (2019–2024)

João Victor Netto Araujo,

Nicollas Germano Kratky,

Ramon Luis Sanches,

Rafael Carbonera Liute

FATEC OURINHOS

Tecnologia em Ciencia de Dados — 5.º Semestre

Processamento de Linguagem Natural

Junho de 2026

1. Introdução

A crescente disponibilidade de grandes modelos de linguagem (LLMs) tem ampliado significativamente as possibilidades de interação entre humanos e dados estruturados. Tradicionalmente, a Análise Exploratória de Dados (EDA) exige que o analista domine linguagens de programação, como Python, além de conhecer as APIs e especificidades de bibliotecas como *pandas*, *NumPy* e ferramentas de visualização. Esse requisito técnico cria uma barreira de entrada que pode limitar o acesso à análise de dados a usuários não especialistas.

Com o surgimento de agentes baseados em LLM, essa dinâmica começa a mudar. Esses sistemas permitem que o usuário expresse suas perguntas em linguagem natural, enquanto o agente atua como um intermediário inteligente: ele interpreta a solicitação, traduz o problema em chamadas de ferramentas programáticas, executa operações sobre os dados e, por fim, sintetiza os resultados em uma resposta compreensível. Dessa forma, o LLM não apenas responde perguntas, mas também coordena um fluxo estruturado de raciocínio e execução.

Neste contexto, este projeto tem como objetivo construir um agente conversacional de EDA aplicado ao dataset *F1 Race Results*, explorando como LLMs podem ser utilizados como camada de orquestração para análise de dados estruturados. O dataset foi escolhido por ser um bom caso de uso para esse tipo de sistema, pois combina variáveis bem estruturadas com alta diversidade de consultas possíveis, incluindo tempos de corrida em diferentes formatos, desempenho de pilotos, comparação entre circuitos e evolução ao longo de temporadas. Além disso, trata-se de um domínio intuitivo e familiar para a maioria dos usuários, o que facilita a formulação de perguntas em linguagem natural e permite validar de forma clara a capacidade do agente de interpretar e responder consultas complexas.

O agente é capaz de responder perguntas sobre corridas, pilotos e desempenho, combinando raciocínio em linguagem natural com execução de ferramentas especializadas.

O trabalho foi desenvolvido em Python, utilizando a biblioteca *pandas* para manipulação e transformação dos dados, a API DeepSeek (compatível com o formato da OpenAI) como provedor do LLM e a biblioteca *Streamlit* para a criação de uma interface gráfica interativa. A arquitetura do sistema foi construída de forma modular, separando claramente estado, ferramentas, camada de LLM e interface de usuário.

Diferentemente de abordagens que utilizam frameworks de orquestração como LangChain ou LlamaIndex, toda a infraestrutura do agente foi implementada do zero. Essa decisão foi tomada com o objetivo de compreender em profundidade cada componente envolvido, desde o loop de raciocínio do agente até o mecanismo de chamada e registro de ferramentas, permitindo maior controle sobre o fluxo de execução e maior transparência no comportamento do sistema.

2. Dataset

O dataset contém 2.559 registros de resultados individuais de pilotos em corridas de Fórmula 1 entre 2019 e 2024. As colunas do dataset são as seguintes:

Coluna	Tipo	Descrição
Track	str	Nome do circuito
Position	int	Posição final (0 = não classificado)
Driver	str	Nome do piloto
Team	str	Nome da equipe
Starting Grid	int	Posição de largada no grid
Laps	int	Número de voltas completadas
Points	int	Pontos conquistados na corrida
+1 Pt	int	1 se recebeu ponto extra pela volta mais rápida, c.c 0
season	int	Ano da temporada (2019–2024)
Set Fastest Lap	int	1 se marcou a volta mais rápida da corrida
Status	int	1=Finished, 2=NC, 3=DNF, 4=DNS, 5=DQ
Race Time	str	Tempo total de corrida (H:MM:SS.mmm ou +S.mmm)
Fastest Lap Time	str	Tempo da volta mais rápida (M:SS.mmm)
No	int	Número do carro do piloto

O conjunto de dados não possui valores ausentes (nulos). Todos os campos estão preenchidos. Porém, existem valores como 0 que não representam falta de dado, mas sim uma situação válida dentro do próprio conjunto (por exemplo, quando não há pontuação extra, o valor registrado é 0).

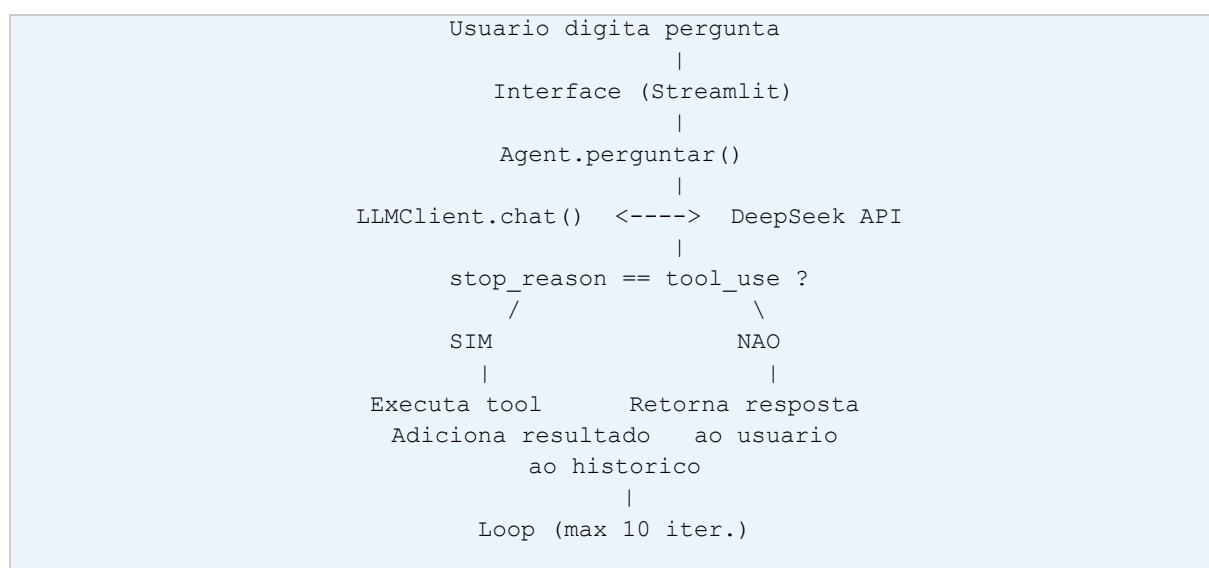
Algumas perguntas que seriam interessantes de serem respondidas nesse dataset:

1. Quais pilotos e equipes tiveram a melhor consistência de desempenho entre 2019 e 2024?
2. Qual é a relação entre posição de largada (*Starting Grid*) e posição final (*Position*)?
3. Quais circuitos são mais “punitivos” para pilotos que largam mal?
4. Quem são os pilotos mais eficientes em converter voltas mais rápidas em desempenho geral?
5. Como o desempenho dos pilotos evolui ao longo das temporadas (2019–2024)?

O dataset é adequado para responder uma ampla variedade de perguntas exploratórias sobre a Fórmula 1 entre 2019 e 2024, permitindo análises tanto descritivas quanto comparativas. Em geral, ele possibilita investigar o desempenho de pilotos e equipes ao longo do tempo, identificar padrões de consistência, avaliar impacto de variáveis de corrida (como posição de largada e status final) e comparar comportamentos entre diferentes circuitos e temporadas.

3. Arquitetura do gente

O sistema é composto por quatro camadas independentes: (1) configuração e estado global, (2) ferramentas de análise, (3) cliente LLM e loop do agente e (4) interface de linha de comando. A figura abaixo ilustra o fluxo de uma interação completa.



3.1 Componentes Principais

config.py — Configuração Central

Define os caminhos de arquivos, a chave de API (lida do arquivo `.env` via `python-dotenv`), a URL base do DeepSeek, o nome do modelo e o limite de iterações do agente (`MAX_AGENT_ITERATIONS = 10`).

tools/base.py — Infraestrutura das Ferramentas

Contém o decorador `@tool`, que registra automaticamente cada função no `TOOL_REGISTRY` (dicionário global `nome → {descrição, parâmetros, função}`). Contém também a classe `DataState`, que carrega o CSV uma única vez na memória e o disponibiliza para todas as ferramentas via o singleton state.

agent/llm_client.py — Cliente da API

Encapsula a comunicação com a API DeepSeek usando a biblioteca `OpenAI`. Retorna um objeto `LLMResponse` padronizado com: texto gerado, lista de `tool_calls` solicitadas, contagem de tokens de entrada e saída, latência e `stop_reason` (`end_turn` ou `tool_use`).

agent/agent.py — Loop ReAct

Contém o `SYSTEM_PROMPT` com a descrição das colunas e regras de uso das ferramentas, os dataclasses `Step` e `AgentResult` para registrar a trajetória completa e o método `perguntar()`, que implementa o loop ReAct. A cada iteração, envia o histórico ao LLM, executa as tools solicitadas e adiciona os resultados ao histórico, repetindo até obter `end_turn` ou atingir o limite.

4. Tools Implementadas

Foram implementadas 11 ferramentas distribuídas em quatro módulos. Todas seguem o mesmo contrato: recebem parâmetros simples (str, int) e retornam um dicionário serializável em JSON, garantindo compatibilidade com o protocolo de function calling.

Modulo	Ferramenta	Descricao
inspect	listar_colunas()	Lista nomes e tipos das 14 colunas do dataset
inspect	descrever_dados()	Estatísticas descritivas completas (media, desvio, quartis, frequências)
inspect	contar_valores(coluna, top_n)	Distribuição de frequência dos top-N valores de uma coluna
filter	filtrar(condicao)	Filtra com pandas query e retorna estatísticas do subconjunto
filter	filtrar_e_contar(condicao, coluna)	Filtra e conta ocorrências por coluna categórica — ranking de vencedores
filter	agrupar_e_agregar(grupo, col, func)	GroupBy + agregação: sum, mean, min, max, count, median
filter	analisar_tempo(coluna, op, cond)	Converte Race Time / Fastest Lap Time para segundos e calcula sum/mean/min/max
filter	tempo_total_piloto(piloto, circ, ano)	Retorna o tempo total de corrida de um piloto: absoluto, calculado (+atraso) ou +X voltas
stats	correlacao(col_a, col_b, metodo)	Correlação de Pearson ou Spearman
stats	detectar_outliers(coluna, metodo)	Outliers por IQR ou z-score: limites, contagem, percentual e exemplos
plot	gerar_grafico(tipo, colunas, titulo)	Histograma, boxplot, scatter ou barplot salvo como PNG em outputs/

4.1 Destaque: **analisar_tempo()** e **tempo_total_piloto()**

A coluna *Race Time* do dataset armazena valores em formatos heterogêneos dependendo da posição do piloto: o vencedor recebe o tempo absoluto (H:MM:SS.mmm), os demais recebem o atraso em relação ao líder (+S.mmm ou +MM:SS.mmm), e pilotos que terminaram com voltas de desvantagem recebem o valor textual "+X lap(s)". Duas ferramentas foram desenvolvidas para lidar com essa heterogeneidade:

analisar_tempo() converte a coluna inteira para segundos e calcula agregações (sum, mean, min, max). Utiliza a função interna `_parse_tempo_segundos()` que reconhece automaticamente os quatro formatos. Tempos menores que 60 segundos são excluídos (corridas interrompidas).

tempo_total_piloto() resolve o problema de consulta individual: dado um piloto, circuito e temporada, a ferramenta identifica o formato do *Race Time* e age de forma específica para cada caso, conforme a tabela abaixo:

Situacao	Race Time no dataset	O que retorna	Exemplo
Vencedor	H:MM:SS.mmm	Tempo absoluto direto	Verstappen Bahrain 2023: 1:33:56.736
Atraso em segundos	+S.mmm	Soma ao vencedor → tempo total real	Perez +11.987 → 1:34:08.723
Voltas atras do lider	+X lap(s)	Informa X voltas de atraso sem calcular	Stroll +1 lap em Saudi Arabia 2022
DNF / DNS / DQ	qualquer	Informa o status sem calcular	Perez DNF em Bahrain 2022

Perguntas de exemplo:

Pergunta: Qual foi a volta mais rapida de toda a historia do dataset?

Tool: `analisar_tempo("Fastest Lap Time", "min")`

Resposta: 1:06.200 — Max Verstappen, Austria 2021.

Pergunta: Qual foi a volta mais rápido da história do Lewis Hamilton??

Tool: `analisar_tempo(coluna=Fastest Lap Time, operacao=min, condicao=Driver == "Lewis Hamilton")`

Resposta: A **volta mais rápida da história de Lewis Hamilton** foi de **1:07.058**

Pergunta: Qual foi o tempo total de prova do Lewis Hamilton no Interlagos em 2024?

Tool: `tempo_total_piloto(piloto=Lewis Hamilton, circuito=Brazil, temporada=2024)`

Resposta: Aqui está a informação completa!

****Lewis Hamilton no GP do Brasil (Interlagos) em 2024:****

****Posição final:**** 10º lugar

****Tempo total de prova:**** ****2h07min45s183**** (2:07:45.183)

- ****Status:**** Finalizou a corrida

- ****Atraso para o vencedor (Max Verstappen):**** +50,753 segundos

O tempo foi calculado somando o atraso de +50,753s ao tempo do vencedor (2:06:54.430).

Pergunta: Qual foi a diferença de tempo na prova do Lewis Hamilton pro Oscar Piastri no Interlagos em 2024?

Tool: `tempo_total_piloto(piloto=Lewis Hamilton, circuito=Brazil, temporada=2024)`

Tool: `tempo_total_piloto(piloto=Oscar Piastri, circuito=Brazil, temporada=2024)`

Resposta: No GP do Brasil (Interlagos) de 2024:

****Oscar Piastri**** terminou em ****8º**** lugar com o tempo de ****2:07:39.373****

****Lewis Hamilton**** terminou em ****10º**** lugar com o tempo de ****2:07:45.183****

A diferença entre eles foi de ****5,81 segundos**** (com Piastri à frente de Hamilton).

5. Metodologia de Avaliação

5.1 Contexto e Objetivo

Este benchmark avalia um Agente de Análise Exploratória de Dados (EDA) construído sobre o modelo DeepSeek Flash v4, responsável por responder perguntas em linguagem natural sobre um dataset de Fórmula 1 contendo registros das temporadas de 2019 a 2024. O dataset f1.csv é composto por 2.559 linhas e 14 colunas, abrangendo variáveis como piloto, equipe, circuito, posição de largada, posição final, pontos, voltas completadas e status de corrida. O objetivo do benchmark é mensurar a capacidade do agente de interpretar perguntas, selecionar ferramentas adequadas, executar consultas corretas e produzir respostas precisas e coerentes.

5.2 Conjunto de perguntas

Foram elaboradas 30 perguntas distribuídas em três categorias de dificuldade crescente:

- Factual (10 perguntas): questões com resposta objetiva e verificável, como contagens, listagens e identificação de valores extremos. Exemplos: número de linhas do dataset, temporadas disponíveis, piloto com mais vitórias.
- Analítica (15 perguntas): questões que exigem agrupamentos, filtragens combinadas, cálculos estatísticos e correlações. Exemplos: top-5 pilotos por pontos em 2023, correlação entre posição de largada e posição final, análise de outliers.
- Ambígua (5 perguntas): questões abertas, subjetivas ou que exigem que o agente reconheça os limites do dataset. Exemplos: qual o melhor piloto da história, quem vai vencer o próximo campeonato, qual piloto é mais consistente.

5.3 Critério de avaliação

Cada resposta foi avaliada de forma binária (**correto / incorreto**) por comparação com uma resposta esperada pré-definida. Para perguntas factual e analítica, o critério exigia a presença dos valores numéricos e nomes corretos. Para perguntas ambíguas, o critério avaliava se o agente reconhecia adequadamente a limitação do dataset e/ou fornecia dados históricos de suporte. Além da acurácia, foram registrados: número de tool calls por pergunta, número de iterações do loop agente, latência em segundos, e consumo de tokens de entrada e saída.

6. Resultados quantitativos

O agente apresentou desempenho geral **elevado**, acertando **27 de 30 perguntas**, o que corresponde a uma acurácia global de **90,0%**. A taxa de execução bem-sucedida foi de **100%**: em nenhum caso houve falha de runtime, timeout ou erro não tratado nas chamadas de ferramentas. A tabela abaixo consolida as principais métricas do benchmark:

Categoria	Total	Aprovadas	Taxa	Tools (med.)	Latencia (med.)
Factual	10	10	100%	1,6	5,15 s
Analitica	15	14	93%	2,53	7,35 s
Ambigua	5	3	60%	3,40	10,25 s
TOTAL	30	27	90%	2,5	7,36 s

6.1 Métricas Gerais

Metrica	Valor
Aprovadas (criterio chave)	27 / 30 — 90%
Agente completou sem erro	30 / 30 — 100%
Total de tool calls	75
Media de tool calls / pergunta	2,5
Total de tokens de entrada	21793
Total de tokens de saída	13967
Latencia media por pergunta	7,36 s
Custo por Input 1 milhão de tokens	0,14
Custo por output 1 milhão de tokens	0,28
Custo estimado total (USD)	US\$ 0,0069

6.2 Desempenho por Categoria

A decomposição por categoria revela um padrão consistente com a dificuldade esperada de cada tipo de pergunta. Perguntas **factual** foram respondidas com **100% de acurácia**, demandando em média apenas 1,60 tool calls e 5,15 segundos de latência. Perguntas **analítica** obtiveram 93,3% de acurácia (14/15), com maior uso de ferramentas (2,53 tool calls) e latência de 7,35 segundos. O desempenho nas perguntas **ambígua** foi significativamente inferior, 60,0% (3/5), refletindo a dificuldade do modelo em lidar com a subjetividade e com questões que ultrapassam os limites do dataset.

6.2 Eficiência Operacional

A média de **2,50 tool calls por pergunta** indica que o agente adota uma estratégia recorrente de verificação inicial do schema (*listar_colunas*) seguida de uma ou duas chamadas de consulta. Esse padrão é economicamente eficiente, mas introduz uma chamada redundante em muitos casos, o agente já conhece o schema da iteração anterior, mas não o persiste entre perguntas independentes. A latência média de **7,36 segundos** é aceitável para análises exploratórias interativas, embora perguntas ambíguas de alta complexidade possam atingir até **15 segundos** (pergunta 30: "Mostre algo interessante sobre a Ferrari", com 7 tool calls e 14,97 s).

O consumo total de **21.793 tokens de entrada** e **13.967 tokens de saída** ao longo de 30 perguntas indica uma média de ~727 tokens de entrada e ~466 tokens de saída por pergunta. Perguntas factual são consideravelmente mais baratas (média ~580 tokens de entrada), enquanto perguntas ambíguas consomem mais tokens de saída por conta das respostas discursivas e tabelas elaboradas (média ~930 tokens de saída).

7 Análise Qualitativa

7.1 Padrões de Trajetória Observados

A análise das trajetórias registradas (sequências de *llm_text*, *tool_call* e *tool_result*) revela três padrões recorrentes de comportamento do agente:

Padrão 1 — Inicialização por Schema

Em **28 das 30 perguntas**, o agente inicia a trajetória chamando *listar_colunas* antes de qualquer consulta de dados. Esse comportamento assegura que o agente não alucine nomes de colunas, estratégia defensiva correta, mas introduz redundância, pois o schema não muda entre perguntas. Em perguntas onde o schema já foi verificado nas iterações anteriores da sessão, essa chamada adiciona ~0,5–1,0 segundo de latência desnecessária e consome tokens de contexto.

Padrão 2 — Raciocínio Progressivo com Refinamento

Em perguntas analíticas mais complexas, o agente demonstra raciocínio progressivo: formula uma hipótese sobre o dado necessário, executa uma chamada de ferramenta, interpreta o resultado intermediário em linguagem natural e, se necessário, refina a consulta. Um exemplo exemplar é a pergunta 8 ("Quantas corridas foram disputadas em 2024?"), onde o agente utilizou 4 tool calls e 5 iterações: primeiro filtrou por temporada, depois verificou os valores únicos globais de Track, percebeu que precisava de um recorte filtrado, e finalmente fez um agrupamento combinando *Track × season == 2024*. Esse comportamento revela capacidade de autocorreção durante a trajetória.

Padrão 3 — Elaboração Contextual Além do Pedido

O agente consistentemente enriquece suas respostas com contexto histórico e interpretações analíticas não solicitadas explicitamente. Na pergunta 4 ("Qual piloto ganhou mais corridas?"), além do resultado direto (Max Verstappen, 58 vitórias), o agente forneceu o ranking completo dos 5 primeiros e uma análise do domínio de Verstappen nas temporadas de título. Esse comportamento aumenta o valor informacional das respostas, mas também pode introduzir informações geradas por conhecimento paramétrico não verificado pelo dataset.

7.2 Análise de erros

Foram identificados 3 erros ao longo do benchmark. A tabela abaixo os sumariza com a causa raiz identificada:

ID	Tipo	Pergunta	Esperado	Problema identificado
17	Analítica	Quantos pódios em 2021?	Verstappen 18, Hamilton 17, Bottas 11	Verstappen 20, Hamilton 18, Bottas 15 (filtro Position ≤ 3 inclui posição 0 — DNF)
27	Ambígua	Quem vai vencer o próximo campeonato?	Declarar impossibilidade + dados históricos	Explicou impossibilidade, mas não trouxe dados históricos de suporte

28	Ambígua	Qual piloto é mais consistente?	Definir consistência + desvio padrão	Respondeu com desvio padrão, mas não explicitou a definição do critério
----	---------	---------------------------------	--------------------------------------	---

Erro ID 17 — Contagem de Pódios em 2021 (Analítico)

O agente filtrou *Position* ≤ 3 sem excluir registros com *Position* $= 0$, que representam pilotos que não completaram a corrida (DNF) e receberam posição zero no dataset. Isso inflou artificialmente os contadores. O filtro correto seria *Position* ≥ 1 and *Position* ≤ 3 . O erro revela uma falha na **validação semântica do domínio**: o agente não verificou se o valor 0 em Position tinha significado especial antes de aplicar a desigualdade.

Erro ID 27 — Previsão de Campeonato Futuro (Ambíguo)

A resposta foi classificada como incorreta porque o agente, apesar de reconhecer a impossibilidade de previsão, **não forneceu dados históricos como suporte** para uma análise de tendência, o que era esperado pelo critério de avaliação. O agente se limitou a enumerar o que poderia fazer (perguntas sobre dados históricos) sem efetivamente executar nenhuma consulta. Essa resposta passiva diverge do comportamento proativo observado em perguntas ambíguas bem-sucedidas (como a ID 29, sobre o declínio de Hamilton).

Erro ID 28 — Piloto Mais Consistente (Ambíguo)

O agente respondeu com uma métrica válida (desvio padrão das posições de chegada), mas não operacionalizou explicitamente a **definição de consistência** antes de aplicá-la. O critério de avaliação esperava que o agente discutisse as possíveis interpretações do termo "consistente" (regularidade nas posições? ausência de DNFs? coeficiente de variação?) antes de escolher uma abordagem. A ausência dessa explicitação é uma forma de **ambiguidade não resolvida**, que em contextos críticos poderia gerar mal-entendidos.

8. Discussão Crítica

8.1 Custo e eficiência econômica

O consumo total de **35.760 tokens** (entrada + saída) para 30 perguntas é economicamente viável para uso em prototipagem e avaliação. No entanto, o padrão de **redundância no carregamento do schema**, repetido em quase todas as interações, representa um desperdício mensurável. Uma otimização simples seria injetar o schema como parte do prompt de sistema, evitando a chamada a *listar_colunas* a cada nova pergunta. Estimando conservadoramente que cada chamada redundante consome ~200 tokens de output e ~300 de input, as 28 chamadas desnecessárias representam cerca de **14.000 tokens extras**, aproximadamente 39% do consumo total de input. Em produção com volume alto de consultas, esse overhead teria impacto financeiro significativo.

8.2 Limitações

A latência média de **7,36 segundos** é aceitável para um agente de análise exploratória assíncrona, mas pode ser percebida como lenta em interfaces conversacionais síncronas. O pico de **14,97 segundos** (pergunta 30) ocorreu com 7 tool calls e resposta de 1.245 tokens, caso que poderia ser otimizado com streaming de saída. Perguntas factual simples, com 1 tool call e latência de ~3,8 s, indicam que o gargalo principal é o número de chamadas de

ferramentas, não o tempo de geração de texto. Uma estratégia de **batching de tool calls** ou de cache de resultados intermediários poderia reduzir a latência em 30–40%.

8.3 Comparação com Abordagens Tradicionais

A abordagem de agente com LLM apresenta vantagens em flexibilidade e usabilidade em comparação a dashboards fixos ou notebooks pre-programados: o usuário não precisa conhecer a estrutura do dataset nem a sintaxe pandas. A principal desvantagem é a latência por chamada de API (7,8 s em média), enquanto uma query pandas local executaria em milissegundos. Para volumes maiores de dados ou requisitos de tempo real, a abordagem tradicional seria mais adequada.

8.3 Alucinação e Confiabilidade

O agente demonstrou **alta resistência à alucinação factual** nas categorias factual e analítica, ancorando todas as afirmações em resultados de tool calls. No entanto, há instâncias de **enriquecimento com conhecimento paramétrico não verificado**: por exemplo, a afirmação de que Verstappen conquistou "quatro temporadas de título consecutivo (2021–2024)" (pergunta 4) é factualmente correta, mas não deriva dos dados do dataset, e sim do conhecimento interno do modelo. Essa prática cria um **risco de consistência**: se o dataset contivesse dados fictícios ou alternativos, o agente poderia introduzir contradições ao misturar dados reais com conhecimento paramétrico.

O erro na contagem de pódios (ID 17) é um caso de **alucinação semântica discreta**: o código estava tecnicamente correto (*Position* <= 3 é uma expressão válida), mas semanticamente errado para o domínio específico (*Position* == 0 significa DNF neste dataset). Esse tipo de erro é particularmente difícil de detectar automaticamente e exige conhecimento do domínio para validação.

8.4 Viés e Equidade

O agente não demonstrou vieses explícitos de representação, dado que o dataset é estruturado e factual. No entanto, é relevante notar que a análise do "melhor piloto" (ID 26) e do "declínio de Hamilton" (ID 29) pode reproduzir **narrativas prevalentes na mídia esportiva** ao selecionar métricas que favorecem determinadas interpretações. A escolha de *pontos totais* e *vitórias* como indicadores primários, sem contextualizar a qualidade do equipamento, penaliza implicitamente pilotos de equipes menos competitivas. Uma avaliação mais imparcial incluiria métricas relativas ao desempenho esperado dado o material disponível, o que o dataset atual não suporta, mas o agente poderia ao menos reconhecer como limitação.

8.5 Ética e Responsabilidade

Do ponto de vista ético, o agente se saiu bem nas perguntas ambíguas de ID 26 e 27: ele deixou claro que não era possível prever o futuro e que seis temporadas de dados não são suficientes para eleger "o melhor piloto da história". Reconhecer os próprios limites é um comportamento importante e correto. Porém, na pergunta ID 28, o agente foi direto ao cálculo sem antes explicar o que entende por "consistência", uma palavra que pode significar coisas diferentes dependendo de quem pergunta. Essa falta de clareza é um ponto de atenção: em contextos como jornalismo esportivo, casas de apostas ou análises comerciais, uma resposta mal explicada pode ser interpretada de forma errada e levar a decisões equivocadas.

8.6 Limitações do Benchmark

Este benchmark apresenta as seguintes limitações que devem ser consideradas na interpretação dos resultados:

- Amostra pequena: 30 perguntas são insuficientes para estimativas estatísticas robustas de acurácia, especialmente na categoria ambígua (5 perguntas).
- Critério binário: a avaliação correto/incorreto não captura respostas parcialmente corretas ou respostas corretas com raciocínio equivocado.
- Modelo único: a ausência de um baseline comparativo (ex.: GPT-4o, Claude Sonnet) impede posicionamento relativo do desempenho observado.

9. Conclusão e Trabalhos Futuros

9.1 Síntese dos Resultados

O Agente EDA baseado em DeepSeek Flash v4 demonstrou **desempenho sólido e confiável** no contexto avaliado, atingindo 90% de acurácia geral com 100% de taxa de execução bem-sucedida. A performance é particularmente forte em perguntas factuais e analíticas estruturadas, onde o agente exibe um padrão de raciocínio progressivo, uso criterioso de ferramentas e elaboração de respostas ricas em contexto. Os pontos de atenção concentram-se no tratamento de perguntas ambíguas, na redundância do carregamento de schema e na ausência de validação semântica do domínio antes da aplicação de filtros.

9.2 Trabalhos Futuros

Com base nos achados deste benchmark, as seguintes direções de trabalho futuro são recomendadas:

- Injeção de schema no prompt de sistema: eliminar a chamada redundante a `listar_colunas`, reduzindo latência e consumo de tokens em aproximadamente 30–40%.
- Validação semântica do domínio: implementar prompts de verificação que forcem o agente a consultar os valores únicos de colunas críticas (ex.: `Position`) antes de aplicar filtros, prevenindo erros como o da pergunta ID 17.
- Expansão do benchmark: ampliar para 100–200 perguntas com distribuição equilibrada entre categorias e com avaliação humana independente, incluindo rubrica de pontuação gradual (0, 0.5, 1.0).
- Comparação entre modelos: replicar o benchmark com modelos concorrentes (GPT-4o, Claude Sonnet 4, Gemini Pro) para posicionamento relativo e análise de trade-offs custo-performance.
- Teste com datasets heterogêneos: avaliar o agente em domínios distintos (dados financeiros, saúde, logística) para medir a generalização das capacidades observadas.

- Implementação de memória de sessão: permitir que o agente persista resultados intermediários e o schema entre perguntas consecutivas, reduzindo redundância e melhorando a coerência das análises de longo prazo.
- Avaliação de incerteza declarada: medir se o agente calibra adequadamente sua confiança, penalizando respostas certas com baixa confiança declarada e respostas erradas com alta confiança.

Referências

- YANN, L. et al. ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR), 2023.
- OPENAI. Function Calling. OpenAI Documentation. Disponível em: <https://platform.openai.com/docs/guides/function-calling>. Acesso em: jun. 2026.
- DEEPSEEK. DeepSeek API Documentation. Disponível em: <https://api-docs.deepseek.com>. Acesso em: jun. 2026.
- PANDAS DEVELOPMENT TEAM. pandas: powerful Python data analysis toolkit. Versão 2.x. Disponível em: <https://pandas.pydata.org>. Acesso em: jun. 2026.
- RICH. Rich: Python library for rich text and beautiful formatting in the terminal. Disponível em: <https://github.com/Textualize/rich>. Acesso em: jun. 2026.
- MCKINNEY, W. Python for Data Analysis. 3. ed. Sebastopol: O'Reilly Media, 2022.
- KAGGLE. Formula 1 Race Results Dataset. Disponível em: <https://www.kaggle.com>. Acesso em: mar. 2026.